Swarm Drones: A Comprehensive Technical Report

> **Context:** Research reference for UAV swarm systems — covering foundational definitions, control architectures, coordination algorithms, bio-inspired methods, coverage strategies, communication approaches, and the role of multi-agent reinforcement learning.
> **Date:** April 2026

---

1. Foundational Definitions

1.1 What Is a Swarm?

A swarm, in the context of unmanned aerial vehicles, is a group of autonomous agents that collectively accomplish a mission through local interactions rather than centralized command. The defining properties of a true swarm are borrowed from biological swarm intelligence — the emergent group behaviors observed in flocks of birds, schools of fish, and colonies of ants — and can be summarized as follows:

Self-organization. Global order arises from local rules. No single agent has a blueprint of the desired group behavior. Instead, each agent follows simple rules involving its immediate neighbors, and the collective pattern — whether a formation, a search sweep, or an adaptive coverage mesh — emerges from the sum of these interactions.

Scalability. A well-designed swarm algorithm works with 5 drones or 500. The individual agent's decision-making depends only on its local neighborhood, so adding more agents does not require redesigning the coordination logic. DARPA's OFFSET program demonstrated this by having a single operator manage over 130 physical drones simultaneously using AI-driven clustering protocols, and Northrop Grumman reported controlling 200 drones in mixed-terrain tests in early 2025 with sub-millisecond command latencies.

Robustness and fault tolerance. Because no single agent is critical, losing one or several drones degrades performance gracefully rather than causing catastrophic failure. L3Harris's AMORPHOUS command-and-control system, unveiled in February 2025, demonstrated this by enabling any drone in a swarm to autonomously assume coordination duties if another asset is lost.

Decentralization. The hallmark of a true swarm is the absence of a single point of failure or a single point of authority. Decisions are distributed. This contrasts sharply with a remotely piloted fleet, where a ground station issues commands to each vehicle individually.

1.2 Multi-Agent System (MAS)

A multi-agent system is the broader computer science abstraction that encompasses swarms. A MAS is any system composed of multiple interacting autonomous agents that perceive their

environment and take actions to achieve individual or collective goals. In a MAS, agents may be cooperative (working toward a shared objective), competitive (pursuing conflicting objectives), or a mixture of both.

A drone swarm is a specific instance of a cooperative MAS in which the agents are physical UAVs operating in shared airspace. The MAS framework provides the theoretical underpinning — the decision-making models, communication protocols, and coordination mechanisms — while "swarm" emphasizes the bio-inspired, emergent, and scalable nature of the group behavior.

The formal mathematical model most commonly used for cooperative multi-agent decision-making under uncertainty is the **Decentralized Partially Observable Markov Decision Process (Dec-POMDP)**. In a Dec-POMDP, each agent receives only a partial observation of the true world state and must select actions based solely on its local observation history. The joint policy of all agents determines the team reward. This formalism captures the core challenge of swarm coordination: agents must cooperate effectively despite having incomplete, potentially stale information about each other and the environment.

1.3 Formation vs. Swarm

These two terms are often conflated but describe different levels of coordination:

Formation refers to a specific geometric arrangement of agents — a V-shape, a line, a grid, a diamond — maintained through active control. Formation control is a sub-problem within swarm robotics. It assumes a desired spatial configuration and asks: how do we get the agents there, and how do we keep them there as conditions change? Formations are typically rigid or semi-rigid: the relative positions between agents are explicitly defined.

Swarm behavior is more fluid. A swarm may adopt formations, dissolve them, split into sub-groups, reform, and adapt its spatial distribution in response to detections, obstacles, or mission changes. Swarming is the superset; formation flying is one possible mode within it. A search-and-rescue swarm, for instance, might use a spread formation during the search phase, switch to a tight cluster when a survivor is found, and then split — some drones maintaining close observation while others resume the search pattern.

1.4 Fleet vs. Swarm

A fleet of drones is a collection of vehicles managed by a centralized operator or ground station, each receiving individual mission commands. There is no inter-agent coordination in the control loop — the intelligence resides entirely in the operator or a central planner. A swarm, by contrast, embeds the coordination intelligence in the agents themselves. The distinction is operational: in a fleet, if the ground station goes down, the drones are lost; in a swarm, they continue.

1.5 Heterogeneous vs. Homogeneous Swarms

A homogeneous swarm consists of identical agents with the same hardware, sensors, and capabilities. This simplifies algorithm design because any agent can substitute for any other. A

heterogeneous swarm contains agents with different payloads — some might carry high-resolution cameras, others thermal sensors, others relay communication nodes. Heterogeneous swarms are more capable but significantly harder to coordinate because task allocation must account for individual capabilities.

---

2. Control Architectures

The control architecture determines where decisions are made and how information flows within the swarm. This is arguably the single most important design choice in any multi-drone system.

2.1 Centralized Control

In a centralized architecture, a single entity — typically a ground control station or a cloud server — collects state information from all drones, computes a global plan, and sends individual commands back to each agent. The central planner has full observability: it sees every drone's position, battery level, sensor readings, and can optimize globally.

Advantages. Global optimality is achievable because the planner has complete information. Coordination is straightforward — there is no ambiguity about who does what. Complex objectives involving trade-offs between agents (e.g., balancing workloads or minimizing total energy) are naturally handled.

Disadvantages. The central planner is a single point of failure. Communication bandwidth scales linearly (or worse) with the number of agents — each drone must send its state and receive its commands. Latency increases as the swarm grows. The computational complexity of planning grows combinatorially with the number of agents, making real-time replanning intractable for large swarms. In contested environments, the communication link to the central node is a vulnerability.

Centralized control is practical for small teams (2–5 drones) in benign environments with reliable communication links. Many commercial drone shows — the synchronized light displays involving hundreds or thousands of drones — use centralized choreography because the environment is controlled, the trajectories are pre-computed, and real-time adaptation is not required.

2.2 Decentralized Control

In a fully decentralized architecture, there is no central planner at all. Each drone makes decisions autonomously based on its own sensor data and information received from its immediate neighbors. There is no global state — each agent has only a local view of the world.

Advantages. No single point of failure. Communication load is distributed and typically bounded by the number of neighbors rather than the total swarm size. The architecture naturally

scales — adding more drones does not increase the computational burden on any existing agent. It is inherently robust to communication disruptions: if an agent loses contact with some neighbors, it can still act on available information.

Disadvantages. Without global information, achieving globally optimal behavior is extremely difficult. Agents may make locally rational decisions that are globally suboptimal — for example, multiple drones may converge on the same target while others are left unattended. Coordination is implicit, relying on well-designed local interaction rules rather than explicit negotiation. Convergence to desired global behaviors may be slow, and formal guarantees are harder to establish.

Decentralized control is the architecture of choice for large swarms (10+ agents), contested environments, and scenarios where communication cannot be guaranteed. The Reynolds flocking model, consensus-based formation control, and Voronoi-based coverage are all inherently decentralized algorithms.

2.3 Hybrid Architectures

Most practical swarm systems use a hybrid approach that blends centralized and decentralized elements. A common pattern is **hierarchical control**: a high-level central planner assigns sub-tasks or regions to groups of drones, and within each group, the drones coordinate in a decentralized manner. This limits the central planner's burden to managing groups rather than individuals, while preserving local autonomy.

Another hybrid pattern is **Centralized Training with Decentralized Execution (CTDE)**, which has become the dominant paradigm in multi-agent reinforcement learning for drones. During a training phase (typically in simulation), agents have access to global state information — every agent's observation, action, and reward — which is used to train a centralized critic or value function. Once trained, each agent executes a purely local policy that depends only on its own observations. The global information is discarded at deployment time. CTDE combines the learning efficiency of centralized approaches with the scalability and robustness of decentralized execution.

Key CTDE algorithm families include:

- **Value decomposition methods** (VDN, QMIX, QTRAN, QPLEX) — these learn a joint Q-function during training and decompose it into per-agent utility functions for decentralized execution. QMIX enforces a monotonicity constraint so that the argmax of each agent's utility equals the argmax of the joint Q-function (the Individual-Global-Max or IGM condition).
- **Centralized critic actor-critic methods** (MADDPG, COMA, MAPPO) — these maintain a centralized critic during training that has access to all agents' observations and actions, while each agent's actor (policy) only uses local observations. MAPPO has emerged as a strong, simple baseline that often matches or exceeds more complex methods.

2.4 Leader-Follower Architecture

Leader-follower is neither strictly centralized nor decentralized — it is a **structured decentralized** architecture. One or more drones are designated as leaders, and the remaining drones are followers. The leader navigates autonomously (either through its own planning or by receiving external commands), and the followers maintain a defined spatial relationship to the leader using local sensing or communication.

Relationship to centralized control: The leader acts as a local, mobile command source, but it does not perform global optimization over all agents. Each follower independently computes its own control inputs to maintain the desired offset from the leader. If the leader fails, the formation breaks unless a succession protocol promotes a new leader.

Relationship to decentralized control: Each follower makes autonomous local decisions (where to fly to maintain its offset), so the execution is decentralized. But the strategic direction — where the whole group goes — is centralized in the leader. This makes leader-follower a **semi-centralized** approach.

Leader-follower is popular in practice because it is simple to implement and intuitively understandable. It is widely used for convoy protection, pipeline inspection, and coordinated mapping flights. Its main weaknesses are the single point of failure in the leader and the lack of true emergent behavior — the followers do not adapt collectively to the environment, they simply track the leader.

Multi-leader variants address the single-point-of-failure issue by designating multiple leaders or by dynamically electing leaders based on position, battery level, or sensor capability. In L3Harris's AMORPHOUS system, for instance, any drone can dynamically assume the coordination role.

2.5 Consensus-Based Control

Consensus-based control is a purely decentralized paradigm inspired by distributed computing. Each drone maintains a state variable (position, velocity, heading, or a mission-level quantity like an auction bid), and through iterative local communication with neighbors, all agents converge to a common value or a coordinated set of values. The mathematical foundation is consensus theory from graph theory: if the communication graph is connected, iterative averaging of neighbors' values converges to the group mean.

Consensus is used for formation keeping (agents agree on a reference frame), rendezvous (agents converge to a common position), task allocation (agents agree on which drone handles which task via auction-like protocols), and time synchronization. It requires no leader and no central coordinator, but does require a connected communication graph.

---

3. Coordination and Coverage Algorithms

3.1 Voronoi Tessellation and Lloyd's Algorithm

Voronoi-based coverage is one of the most elegant and widely used decentralized coordination methods for area monitoring. Given a set of generator points (the drone positions), a Voronoi tessellation partitions the plane into cells, one per drone, such that every point in a cell is closer to its generator than to any other. Each drone is "responsible" for its Voronoi cell.

Lloyd's algorithm iteratively improves the tessellation: in each step, every drone computes the centroid of its Voronoi cell and moves toward it. This produces a **Centroidal Voronoi Tessellation (CVT)**, where each generator coincides with its cell's centroid. A CVT minimizes a cost function related to the expected distance between any point in the area and the nearest drone — in other words, it optimizes coverage.

The beauty of Lloyd's algorithm is that it is fully decentralized: each drone only needs to know the boundary of its own Voronoi cell, which requires only neighbor positions, and it independently moves toward its own centroid. No central planner is needed.

Weighted Voronoi partitioning extends the basic approach by introducing a density function over the area. The centroid computation integrates over this density, causing drones to concentrate in high-density (high-importance) regions. This is the foundation for event-driven adaptive coverage: when a detection occurs at a location, the density at that point increases, and the Voronoi centroids shift to pull drones toward it. The density function can encode detection confidence, event priority, temporal decay, and other task-specific quantities.

Recent extensions include Gaussian-distribution-based CVT (SwarmCVT, 2024), which uses Gaussian basis functions for path planning of very large-scale robot swarms, and hybrid Voronoi-genetic algorithm approaches for wireless sensor network deployment in GPS-denied environments.

3.2 Static Grid Coverage

The simplest area coverage strategy is to divide the monitoring area into a fixed grid of cells and assign one drone to each cell's center. The drone hovers or loiters at its assigned position, providing continuous surveillance of its cell.

Advantages. Trivial to implement. Uniform coverage is guaranteed if the number of drones matches the number of cells. Predictable behavior makes it easy to analyze and debug.

Disadvantages. Completely static — no adaptation to detections or events. If the area of interest shifts, the grid cannot respond. Inefficient use of resources: drones hover over empty areas just as long as they hover over high-activity areas. Does not scale well when the area is much larger than what the available drones can cover.

Static grid serves primarily as a baseline for comparison in experimental evaluations. It represents what happens when coordination intelligence is set to zero.

3.3 Lawnmower (Boustrophedon) Coverage

The lawnmower or boustrophedon pattern is a systematic sweep strategy: the area is divided into strips, and each drone flies back and forth along its assigned strip, like mowing a lawn. When a drone reaches the end of one pass, it shifts to the adjacent unswept strip and reverses direction.

Advantages. Guarantees complete coverage if given enough time. The path is predictable and can be pre-computed, simplifying trajectory planning and energy estimation. Well-suited for applications like agricultural spraying, photogrammetric mapping, and environmental surveys where the goal is exhaustive coverage rather than rapid response to events.

Disadvantages. No adaptability. Coverage is temporally sequential — a given area is covered only when the drone's sweep reaches it, which can introduce significant latency for time-sensitive detection. Multi-drone lawnmower coordination requires careful strip allocation to avoid overlap and gaps. For irregular areas or areas with obstacles, the pattern becomes complex.

The lawnmower pattern is widely used in commercial drone mapping and agricultural applications because it pairs naturally with photogrammetric processing, which requires overlapping parallel passes.

3.4 Random Waypoint Patrol

In random waypoint patrol, each drone independently selects a random position within the monitoring area, flies to it, dwells briefly, and then selects a new random position. This is the simplest possible patrol strategy beyond static hovering.

Advantages. Requires zero coordination. Extremely simple to implement. Provides a useful lower bound on performance for comparison with coordinated methods.

Disadvantages. No guarantees on coverage uniformity, detection latency, or efficiency. Drones may cluster randomly in some areas while leaving others unmonitored. No adaptation to detections. Significant overlap and redundancy.

Random patrol is used exclusively as an experimental baseline, not as a practical coverage strategy.

3.5 Bio-Inspired Algorithms

Bio-inspired algorithms draw on the collective behaviors observed in nature — bird flocking, ant foraging, firefly synchronization, fish schooling — to design swarm coordination rules. These algorithms share a common philosophy: simple local rules produce complex, adaptive global behavior.

3.5.1 Reynolds Flocking

Craig Reynolds' 1987 model of flocking is the foundational bio-inspired swarm algorithm. Each agent follows three rules:

1. **Separation:** steer away from neighbors that are too close, to avoid collisions.
2. **Alignment:** steer toward the average heading of nearby neighbors, to match the group direction.
3. **Cohesion:** steer toward the average position of nearby neighbors, to stay with the group.

These three rules, applied locally by each agent with no global knowledge, produce realistic flocking behavior — the kind of coherent, collision-free group motion seen in starling murmurations. Reynolds flocking has been validated on real hardware with up to 30 autonomous drones flying outdoors with collective obstacle avoidance and no central control. The key challenge is tuning the three gain parameters and the neighborhood radius to produce stable, collision-free flocking in the physical constraints of real quadrotors (limited turning rates, communication delays, GPS noise).

Olfati-Saber's 2006 extension of Reynolds' rules added formal stability analysis and a navigational feedback term (a virtual leader or waypoint) that allows the flock to be steered while maintaining cohesion — bridging the gap between pure emergent flocking and goal-directed swarming.

3.5.2 Particle Swarm Optimization (PSO)

PSO, originally developed by Kennedy and Eberhart in 1995, is a metaheuristic optimization algorithm inspired by the social behavior of bird flocking and fish schooling. A population of candidate solutions ("particles") moves through the search space, and each particle adjusts its velocity based on two attractors: its own best-known position (cognitive component) and the swarm's best-known position (social component).

In drone applications, PSO is used for path planning, target search, sensor deployment optimization, and coverage control. Each drone is treated as a particle, and the objective function encodes the mission metric (e.g., maximize coverage, minimize time to detect a target). The drone adjusts its waypoint based on its own best observation and the best observation reported by its neighbors.

PSO's strength is its simplicity and ability to find good solutions in continuous search spaces without gradient information. Its weakness is susceptibility to premature convergence to local optima, especially in high-dimensional or deceptive landscapes. Variants like adaptive PSO, chaotic PSO, and hybrid PSO-Q-learning aim to address this.

3.5.3 Ant Colony Optimization (ACO)

ACO is inspired by the foraging behavior of ants, which deposit pheromone trails that guide other ants toward food sources. In the drone context, ACO is used primarily for path planning

and coverage path planning. Drones leave virtual pheromone trails on visited cells; unvisited or stale-pheromone cells are more attractive, naturally distributing the swarm across the area.

Multi-UAV cooperative coverage using ACO with Q-learning adaptive strategies has been demonstrated for complex agricultural environments, balancing exploration and convergence through uneven pheromone distribution.

3.5.4 Grey Wolf Optimization (GWO)

GWO mimics the social hierarchy and hunting behavior of grey wolves. The three best solutions are designated alpha, beta, and delta wolves, and the rest of the pack (omega wolves) update their positions based on the positions of these leaders. In UAV search-and-rescue, GWO has been combined with thermal confidence maps to guide swarm exploration toward likely survivor locations.

3.5.5 Artificial Potential Fields (APF)

APF treats the target as an attractive potential well and obstacles (including other drones) as repulsive potential fields. Each drone follows the negative gradient of the combined potential field. This produces smooth, collision-free paths toward goals.

APF is intuitive and computationally cheap, but suffers from local minima (where attractive and repulsive forces cancel out, trapping the drone). Various solutions exist: adding random perturbations, virtual waypoints, or combining APF with other methods like RRT* or PSO to escape local minima.

---

4. Formation Control Methods

Formation control is the specific sub-problem of getting and keeping a group of drones in a defined geometric pattern. The major approaches are:

4.1 Leader-Follower

As described in Section 2.4, one drone leads and others maintain fixed offsets. Implementation typically uses PID or sliding-mode controllers on the error between the desired and actual offset from the leader.

4.2 Virtual Structure

The entire formation is treated as a single rigid body. A virtual reference point (the "virtual structure") moves along the desired trajectory, and each drone controls its position relative to this virtual frame. The virtual structure approach simplifies formation keeping to a tracking problem for each individual drone, but does not naturally handle inter-agent collision avoidance and is less flexible for dynamic formation changes.

4.3 Behavior-Based

Each drone runs multiple behaviors simultaneously — goal seeking, obstacle avoidance, formation keeping, collision avoidance — and a weighting or priority scheme blends them. This approach is inspired by Brooks' subsumption architecture in robotics. It is flexible and can handle dynamic environments, but the interaction between behaviors can produce unpredictable emergent effects, and formal stability analysis is difficult.

4.4 Consensus-Based Formation

Agents communicate with neighbors and iteratively converge to a desired formation through consensus on relative positions. This is the most theoretically grounded decentralized formation control method, with provable convergence results under conditions on the communication graph (connectivity) and update rates. It naturally handles switching communication topologies and can incorporate time delays.

4.5 Graph-Based Representations

The communication and control relationships in a formation are naturally modeled as a graph: drones are vertices, and edges represent either communication links or control dependencies. The graph Laplacian encodes the topology and is the key mathematical object in consensus-based formation control. Eigenvalues of the Laplacian determine convergence speed and robustness to link failures.

---

5. Multi-Agent Reinforcement Learning for Swarm Control

MARL has become the dominant research paradigm for learning swarm behaviors that are too complex to hand-design. Rather than writing explicit rules for coordination, a team of agents learns cooperative policies through trial-and-error interaction with a simulated environment.

5.1 The CTDE Paradigm

As introduced in Section 2.3, Centralized Training with Decentralized Execution is the standard training framework. The two main families are:

Value function factorization — VDN (simple additive decomposition), QMIX (monotonic mixing network), QTRAN (transformation-based), QPLEX (duplex dueling decomposition). These methods learn a joint team Q-function during training and factor it into per-agent Q-functions for execution, subject to the IGM condition.

Centralized critic methods — MADDPG (multi-agent DDPG with centralized critic), COMA (counterfactual multi-agent policy gradients), MAPPO (multi-agent PPO with shared centralized value function). MAPPO in particular has shown surprising effectiveness despite its simplicity, often matching or exceeding more complex methods across a range of cooperative benchmarks.

5.2 Key Algorithms in Detail

QMIX uses a mixing network that takes per-agent Q-values and the global state as input and produces the joint Q-value. The mixing network's weights are constrained to be non-negative, enforcing the monotonicity requirement of IGM. This means each agent can independently maximize its own Q-value during execution, and the resulting joint action will also maximize the joint Q-value. The limitation is that this monotonicity constraint prevents QMIX from representing value functions where one agent's optimal action depends non-monotonically on another agent's action.

MAPPO applies Proximal Policy Optimization in a multi-agent setting. Each agent has its own policy network (actor) that maps local observations to actions, and a shared value network (critic) that takes the global state as input. During training, the centralized critic provides stable value estimates for policy updates. During execution, only the local actors are used.

MADDPG extends DDPG to multi-agent settings by training a centralized critic for each agent that takes all agents' observations and actions as input. This allows each agent's critic to model the behavior of other agents, reducing the non-stationarity problem inherent in multi-agent learning.

5.3 Independent Learning (DTE)

The simplest multi-agent RL approach is to treat each agent as an independent learner running its own single-agent RL algorithm (e.g., independent Q-learning, independent PPO). Each agent ignores the existence of other agents and treats them as part of the environment. This is fully decentralized during both training and execution.

Independent learning is surprisingly effective in some settings but fundamentally limited by non-stationarity: from any single agent's perspective, the environment is constantly changing because the other agents are simultaneously updating their policies.

5.4 Communication-Learned Policies

A growing area of research explores agents that learn not just what actions to take, but also what messages to send to their neighbors. Architectures like CommNet, TarMAC, and Graph Neural Network-based approaches (ModGNN, D-GAT) allow agents to learn communication protocols end-to-end alongside their decision policies. The recent DG-MAPPO framework (2025) achieves fully distributed MARL by replacing centralized critics with distributed graph attention networks that infer global state through multi-hop peer-to-peer communication.

---

6. Communication Approaches

Communication is the connective tissue of any multi-agent system. The choice of communication approach has profound implications for scalability, latency, reliability, and the

types of coordination algorithms that can be used.

6.1 Communication Topologies

Full mesh (all-to-all): Every drone communicates directly with every other drone. Provides maximum information sharing but communication overhead scales as $O(n^2)$, making it impractical for large swarms. Suitable only for small teams (3–6 agents).

Star (centralized): All drones communicate through a central hub (ground station or relay drone). Simple routing but creates a bottleneck and a single point of failure. This is the topology implicitly assumed by centralized control architectures.

Ring: Each drone communicates only with its two immediate neighbors in a ring structure. Low bandwidth requirements but slow information propagation (messages must traverse half the ring on average to reach the most distant agent). Fragile — a single link failure breaks the ring.

Graph-based (ad hoc mesh): Each drone communicates with neighbors within communication range. The topology is dynamic — as drones move, links form and break. This is the most realistic model for swarm communication and the one assumed by most decentralized algorithms. The connectivity of the graph (whether every agent can reach every other agent through some chain of links) is a critical requirement for consensus-based algorithms.

Hierarchical: The swarm is organized into clusters, each with a cluster head. Intra-cluster communication is local and frequent; inter-cluster communication passes through the cluster heads. This reduces total communication volume while maintaining global connectivity. It naturally fits the hierarchical control architecture described in Section 2.3.

6.2 Communication Protocols and Technologies

Wi-Fi (802.11): Widely used in research and commercial drones. Provides high bandwidth (tens to hundreds of Mbps) over moderate ranges (100–300m in open air). Supports ad hoc mesh networking. Susceptible to interference in congested RF environments.

Zigbee (802.15.4): Low-power, low-bandwidth protocol designed for sensor networks. Suitable for transmitting telemetry and small control messages. Very low power consumption. Limited bandwidth (250 kbps) makes it unsuitable for streaming camera feeds.

Bluetooth / BLE: Short-range, low-power. Used primarily for ground-to-drone configuration, not for inter-drone swarm communication due to limited range.

LoRa / LoRaWAN: Long-range (several kilometers), very low bandwidth. Suitable for sparse telemetry updates (position, battery level) in wide-area operations. Not suitable for real-time coordination requiring frequent updates.

5G / LTE: Cellular networks provide wide-area coverage with relatively high bandwidth and low latency. Emerging as a viable backhaul for drone swarms in urban environments, especially for

connecting the swarm to cloud-based coordination services. Dependent on cellular infrastructure availability.

Custom radio links (FHSS, UWB): Military and high-reliability applications often use frequency-hopping spread spectrum or ultra-wideband radios for jamming resistance, precise ranging, and secure communication.

MAVLink: Not a physical protocol but a lightweight messaging protocol standard for drone communication. MAVLink defines message types for telemetry, commands, mission items, and parameter management. It runs over any transport layer (serial, UDP, TCP). MAVLink is the communication protocol used by PX4 and ArduPilot flight stacks and is the de facto standard for ground-station-to-drone communication.

6.3 Communication Challenges in Swarms

Bandwidth constraints: If each drone is streaming video (even at low resolution, say 320×240 at 10 fps), the aggregate bandwidth for a 10-drone swarm is significant. Sharing raw images between drones is generally infeasible in practice. This motivates approaches like sharing compressed feature maps, detection results (class + confidence + position — a few bytes per detection), or aggregated model weights (federated learning) instead of raw sensor data.

Latency: For real-time coordination — reactive formation changes, collision avoidance, event-triggered reconfigurations — end-to-end communication latency must be in the tens of milliseconds. Multi-hop routing in mesh networks introduces per-hop delay. The faster the drones are moving, the tighter the latency requirement.

Dynamic topology: As drones move, communication links form and break. Algorithms must be robust to intermittent connectivity. Consensus algorithms, for example, can tolerate temporary disconnections as long as the communication graph is connected on average over time (a property called "jointly connected").

Packet loss and interference: In cluttered environments (urban canyons, forests, indoor), multipath effects and physical obstructions cause packet loss. Swarm protocols must handle lost messages gracefully — either through redundancy, acknowledgments, or designing algorithms that degrade smoothly under information loss.

Security: Swarm communications can be intercepted, jammed, or spoofed. The SwarmRaft framework (2025) from Skolkovo Institute addresses this by adapting the Raft consensus algorithm from distributed systems to enable drones to agree on state updates in GNSS-denied or adversarial environments, using blockchain-inspired consensus for positioning and data integrity.

6.4 Information Sharing Strategies

What information drones share is as important as how they share it:

Position sharing: The minimum information exchange. Each drone broadcasts its GPS coordinates and velocity. Sufficient for basic formation keeping, Voronoi computation, and collision avoidance.

Detection sharing: When a drone detects an object, it broadcasts the detection class, confidence, and estimated world position. This enables event-driven coordination without sharing raw imagery.

Feature map sharing: Each drone extracts intermediate neural network features from its camera feed and shares compressed feature vectors with neighbors. This enables collaborative perception — fusing information from multiple viewpoints — without the bandwidth cost of sharing full images.

Model weight sharing (federated learning): Drones periodically share their locally trained model weights, which are aggregated (e.g., FedAvg) to improve the global model without centralizing raw data. This is relevant for scenarios where drones encounter different environments and conditions, and the diversity of their local data improves the shared model.

Implicit communication: In some MARL approaches, agents communicate indirectly through their observable actions. A drone that breaks formation and flies toward a location implicitly signals to its neighbors that it has detected something, without any explicit message. Some learned communication protocols fall in this category — agents learn to encode information in their observable state.

---

7. Coverage Path Planning (CPP) Algorithms

Beyond the basic patterns described in Section 3, several more sophisticated CPP approaches are used for multi-drone missions:

7.1 Boustrophedon Decomposition

The area is decomposed into convex sub-regions using vertical cell decomposition. Within each sub-region, the drone flies a back-and-forth lawnmower pattern. The decomposition ensures that the pattern can cover non-convex areas and areas with obstacles. Multi-drone coordination assigns different sub-regions to different drones, often using graph-based methods to minimize traversal overlap.

7.2 Spanning Tree Coverage (STC)

The area is discretized into a grid, a spanning tree is constructed over the grid cells, and the drone follows the circumnavigation of the tree. This guarantees complete coverage without revisiting any cell. For multiple drones, the tree can be partitioned into sub-trees, one per drone. The challenge is constructing balanced sub-trees for equitable workload distribution.

7.3 Multi-UAV CPP with In-Flight Replanning

A recent trend (2024–2025) is coverage path planning that can adapt in real time — replanning when a drone fails, when new priority regions emerge, or when environmental conditions change. These approaches typically combine initial decomposition-based plans with online replanning triggered by events.

---

8. Task Allocation

Task allocation answers: which drone does what? In a swarm context, this is a distributed optimization problem.

8.1 Auction-Based Approaches

Drones bid on tasks based on their estimated cost (distance, battery, capability match). A consensus round determines the winning bids. The CBBA (Consensus-Based Bundle Algorithm) and its variants are widely used for distributed task allocation in multi-drone systems.

8.2 Market-Based Approaches

Similar to auctions but with richer negotiation: drones can trade tasks after initial allocation. If drone A finds that task 3 is closer to drone B's current position, they can negotiate a swap. This produces more globally efficient allocations through local trades.

8.3 Learning-Based Allocation

Recent work uses MARL to learn task allocation policies end-to-end. Rather than hand-designing auction rules, the agents learn which tasks to take on through reward-shaped training. The IRADA framework (2025) demonstrated energy-aware reward modulation for persistent monitoring task allocation.

---

9. Energy and Battery Considerations

Battery life is the fundamental physical constraint on drone swarm operations. A typical multirotor drone has 20–40 minutes of flight time, and this is consumed faster in wind, at higher speeds, or with heavier payloads.

Energy-aware swarm algorithms model battery as a depleting resource and integrate it into the coordination logic. Key strategies include dynamic task handoff (a low-battery drone transfers its surveillance task to a charged replacement before returning to the charging station), energy-equitable task allocation (distributing workload to equalize depletion rates), and energy-as-objective optimization (jointly minimizing mission time and energy consumption).

Energy consumption in a multirotor is dominated by hovering and maneuvering — it is approximately proportional to flight time plus a component proportional to the cube of velocity for forward flight. In experimental work, total distance traveled is commonly used as an energy proxy because it is easy to measure from position logs and correlates well with actual energy use.

---

10. Current Trends and Open Problems (2025–2026)

Scalability. Most published experimental results involve 3–10 drones. Scaling to 50–200 drones with physics-based simulation (not just point-mass models) remains computationally challenging. Hierarchical architectures and distributed MARL (DG-MAPPO) are promising directions.

Sim-to-real transfer. Policies trained in simulation often fail on real hardware due to sensor noise, communication delays, and aerodynamic effects not captured by the simulator. Domain randomization, system identification, and progressive transfer are active areas of research.

Heterogeneous swarms. Most algorithms assume identical drones. Coordinating teams with different sensor payloads, flight characteristics, and communication capabilities requires new formulations.

Semantic understanding. Moving beyond binary detection (found/not found) to class-aware, confidence-weighted swarm responses — where the type and certainty of a detection drives different coordination behaviors — is an emerging frontier.

GNSS-denied and adversarial operation. Drone swarms that can operate in GPS-denied environments using visual odometry, inertial navigation, and peer-to-peer consensus for positioning are increasingly important for military and disaster-response applications.

Regulatory integration. As drone swarms move from research to deployment, integration with air traffic management systems (UTM — unmanned traffic management) and compliance with aviation regulations (FAA Part 107, EASA rules) become critical challenges.

References and Further Reading

- Reynolds, C.W. (1987). "Flocks, herds and schools: A distributed behavioral model." ACM SIGGRAPH.
- Olfati-Saber, R. (2006). "Flocking for multi-agent dynamic systems: Algorithms and theory." IEEE Trans. Automatic Control.
- Cortés, J. et al. (2004). "Coverage control for mobile sensing networks." IEEE Trans. Robotics and Automation.
- Du, Q., Faber, V., Gunzburger, M. (1999). "Centroidal Voronoi tessellations." SIAM Review.

- Chung, S.J. et al. (2018). "A survey on aerial swarm robotics." IEEE Trans. Robotics.
- Amato, C. (2024). "An Introduction to CTDE in Cooperative MARL." arXiv:2409.03052.
- Rashid, T. et al. (2018). "QMIX: Monotonic value function factorisation for deep MARL." ICML.
- Yu, C. et al. (2022). "The surprising effectiveness of PPO in cooperative multi-agent games." NeurIPS.
- Alqudsi, Y. & Makaraci, M. (2025). "UAV swarms: Research, challenges, and future directions." J. Eng. Applied Science.
- MDPI Drones (2025). "A Survey on Multi-UAV Path Planning: Classification, Algorithms, Open Problems." Drones 9(4).
- Vásárhelyi, G. et al. (2018). "Optimized flocking of autonomous drones in confined environments." Science Robotics.
- Dev, K. et al. (2025). "SwarmRaft: Consensus for robust drone swarm coordination in GNSS-degraded environments." arXiv.
- Mochurad, L. et al. (2025). "Decentralized drone swarm coordination using RL and parallel computing." Int. J. Intelligent Eng. Systems.

Report prepared April 2026 as a reference document for the Adaptive Area Monitoring research project.